

Oblig 2 – IDG2100

API specifications

Written by Tuva K. Hansen

Table of Contents

<i>Spanish poker dice platform API</i>	3
Author:	3
Problem Statement:.....	3
Solution:	3
Implementation:.....	3
<i>API specification</i>	5
Authentication:.....	5
Endpoints:	6
Resource 1: Users	6
Resource 2: Matches.....	7
Resource 3: Tournaments	8
Resource 4: Comments.....	9
Resource 5: Game Category	9
Resource 6: Leaderboard	10
Resource 7: Activity	10
Resource 8: Trohpy	10

Spanish poker dice platform API

Author:

Tuva Konstanse Hansen

Problem Statement:

My Spanish Poker Dice game lacks a backend, so it's not functional as a real multiplayer platform yet. To make it work as a real game, a REST API is needed to host games, manage users and comments, track matches, organize tournaments, display leaderboards and track game activity.

Without this backend, the frontend has no way to store data, manage users, or run tournaments, so it's important that I get this done.

Solution:

The solution relies on a client-server architecture. The backend is built as a local REST API using Node.js and Express.js, with data stored in a local MongoDB database using Mongoose for schema definition and validation.

I designed MongoDB schemas for six collections:

- Users
- Matches
- Tournaments
- GameCategories
- Comments
- Trophies

I also implemented all necessary API endpoints to handle user management (supporting admin, registered, and anonymous users), match and tournament creation (including result recording), ELO rating updates, leaderboards with sorting options, comment sections, and platform activity tracking.

The API enforces role-based access control, input validation, and rate limiting on all endpoints.

Implementation:

My API solution is structured in layers with routes, controllers, validators, and services, to keep business logic decoupled from HTTP handling. Routes define the URLs and point to the right controller, controllers handle the request and response, but all the actual logic, like calculating ELO ratings after a match, pairing players randomly in tournament brackets, and computing win percentages for the leaderboard, gets handled by the services. In addition, the validators run before the controller, using express-validator, making sure incoming data is valid before anything gets processed.

I also defined MongoDB schemas with Mongoose and created one model file per collection. They all have built-in validation, default values, and hooks that automatically generate IDs and hash passwords on save.

To set user roles I created a simple middleware that reads the X-User-Role header on every request and uses it to decide what the caller is allowed to do. This is based on if the user is anonymous, a registered user or an admin. I also applied rate limiting globally, allowing a maximum of 100 requests per 15 minutes to prevent abuse, but this is added to my server file, and not as a middleware file.

API specification

Authentication:

To access protected endpoints, each request needs an X-User-Role header with the value anonymous, user, or admin. Middleware reads this header and attaches the role to the request, allowing endpoints to restrict access based on user type.

- Anonymous users can access public endpoints such as leaderboards and platform activity, but cannot create matches, join tournaments, or leave comments.
- Registered users can access most endpoints.
- Admin users can create tournaments, ban users, and delete comments.

A problem with this approach, is that anyone could just set X-User-Role: admin and claim to be an admin. Instead, in a fully implemented version, I would replace this with a JSON Web Token. This means that when a user logs in, the server generates a token signed with a secret key, containing the user's role and ID. The token is then sent with every request, and since only the server knows the secret key, it can verify that the token is genuine and hasn't been tampered with. This will make it impossible for users to just claim to be an admin. But for this assignment I kept the simpler approach since full authentication is not required.

Errors:

When something goes wrong, the API returns an appropriate HTTP status code along with a JSON error response containing a message describing the issue.

For example, if a user sends incomplete data, the API returns a 400 with a message explaining which fields are missing. If a user tries to access an admin-only endpoint without the correct role, a 403 is returned.

HTTP status code	Error code	Description
400	BAD_REQUEST	Required fields missing, or invalid input data
401	UNAUTHORIZED	Missing or invalid X-User-Role header
403	FORBIDDEN	User does not have permission for this action
404	NOT_FOUND	Requested resource does not exist
409	CONFLICT	Resource already exists (e.g. username taken)
429	TOO_MANY_REQUESTS	Rate limit exceeded
500	INTERNAL_SERVER_ERROR	Unexpected server error

Endpoints:

Resource 1: Users

URI (verb and route)	Description	Inputs	Outputs
POST /users	Register a new user	Body: username, email, password, age, role (optional, defaults to 'user')	201 with created user object (id, username, email, age, eloRating, role)
POST /users/login	Login with an existing user	Body: username, password	200 with user object (username, email, age, eloRating, role, trophies)
GET /users/:userId	Get a user's profile	Route: id	200 with user object (username, email, age, eloRating, ratingChange, trophies, recentGames, totalGames, ratingChange)
PUT /users/:userId	Update your own profile	Route: id, Header: x-user-role: user, Body: email, password, age (all fields are optional)	200 with updated user object (username, email, age, eloRating, ratingChange, role, trophies, banned)
GET /users	Admin: List all users	Header: x-user-role: admin, Query: page, limit, search (all optional)	200 with paginated array of user objects (userList, totalUsers, page, limit, totalPages) where userList is an array of user objects each containing (username, email, age, eloRating, role, trophies, banned)
PUT /users/:userId /ban	Admin: Ban a user	Route: id, Header: x-user-role: admin	200 with updated user object (username, email, age, eloRating, role, trophies, banned: true)

Resource 2: Matches

URI (verb and route)	Description	Inputs	Outputs
POST /matches	Create new match	Header: x-user-role: user Body: gameId, players (optional, matches can start without players assigned)	201 with created match object (id, players, gameId, status, isAnonymous)
PUT /matches/:matchId/record	Record match result	Route: id Header: x-user-role: user Body: winner, rolls (optional), holds (optional), outcome	200 with updated match object (players, winner, outcome, rolls, holds, status, gameId)
GET /matches/:matchId	Get a specific match	Route: id	200 with match object (players, rolls, holds, outcome, winner, timestamps, gameId)
GET /matches	List all matches	Query: page, limit, status, gameId, userId (all optional)	200 with paginated array of match objects (matchList, totalMatches, page, limit, totalPages)
POST /matches/queue	Join the matchmaking queue	Header: X-User-Role: user, Body: gameId, gameId	200 with match object or waiting status (status, match)
DELETE /matches/queue	Leave matchmaking queue	Header: X-User-Role: user, Body: gameId	200 with success message (message)
POST /matches/:matchId/join	Join an existing waiting match as a registered or anonymous user	Route: matchId, Body: gameId (optional, omit for anonymous users)	200 with updated match object (players, anonymousCount, status, gameId)

Resource 3: Tournaments

URI (verb and route)	Description	Inputs	Outputs
POST /tournaments	Create a tournament	Header: x-user-role: admin Body: title, description, date, breaks (optional), numberOfRounds, trophy (optional), gameCategoryId	201 with created tournament object (title, description, date, status, participants, gameCategory, numberOfRounds, breaks, trophy)
POST /tournaments/:id/join	Join a tournament	Route: id Header: x-user-role: user Body: userId	201 with updated tournament object (participants)
GET /tournaments/:tournamentId	Get tournament	Route: id	200 with tournament object (title, description, date, status, participants, rounds, trophy)
GET /tournaments	List all tournaments	Query: page, limit, status (all optional)	200 with paginated array of tournament objects (tournamentList, totalTournaments, page, limit, totalPages)"
PUT /tournaments/:id/advancedRounds	Advance tournament round	Route: id Header: x-user-role: admin	200 with updated tournament object (rounds, status)

Resource 4: Comments

URI (verb and route)	Description	Inputs	Outputs
POST /comments	Post a comment	Header: x-user-role: user Body: targetId, targetType, comment, userId	201 with created comment object (targetId, targetType, comment, userId, createdAt)
DELETE / comments /:commentId	Delete a comment	Route: id Header: x-user-role: admin	200 with confirmation message
GET / comments	List all comments	Header: x-user-role: admin Query: targetId, targetType, page, limit, search (all optional)	200 with paginated array of comment objects (commentList, totalComments, page, limit, totalPages)

Resource 5: Game Category

URI (verb and route)	Description	Inputs	Outputs
GET /gamecategories/	List all 18 game categories		200 with array of all 18 game category objects (id, numberOfRounds, gameRules, timeController)
GET /gamecategories/:gameCategoryId	Get a specific game category	Route: id	200 with game category object (id, numberOfRounds, gameRules, timeController)

Resource 6: Leaderboard

URI (verb and route)	Description	Inputs	Outputs
GET /leaderboards	Get rankings (filterable by category)	Query: category (gameCategoryId), numberOfRounds, gameRules, timeController, sortBy (wins/winPercentage/matches), page, limit (all optional)	200 with paginated array of ranked users (userList, total, page, limit, totalPages) where each user has (username, eloRating, wins, winPercentage, totalMatches)

Resource 7: Activity

URI (verb and route)	Description	Inputs	Outputs
GET /activities	Get platform activity	None	200 with object (ongoingMatches, activeUsers, last10Matches)

Resource 8: Trophy

URI (verb and route)	Description	Inputs	Outputs
POST /trophies	Create a trophy with image	Header: x-user-role: admin Body: title, tournamentId, image (file upload)	201 with created trophy object (id, title, image, tournamentId)